

UNIVERSIDAD PRIVADA DE TACNA
FACULTAD DE INGENIERÍA
Escuela Profesional de Ingeniería de Sistemas



**Proyecto: “Sistema web de detección y análisis de anomalías en
operaciones de bases de datos NoSQL, 2026”**

Documento de Especificación de Requerimientos de Software (SRS)
Versión 1.0

Curso: BASE DE DATOS 2
Docente: PATRICK JOSE CUADROS QUIROGA

Integrantes:
MANGO PUMA, Franco Teddy
CALDERON RODRIGUEZ, Juan Carlos

Tacna - Perú
2026

**Sistema web de detección y análisis de anomalías en operaciones de bases de datos NoSQL,
2026**

Documento de Especificación de Requerimientos de Software

Versión 1.0

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPFT / CRJC	Equipo de desarrollo	Pendiente de validación académica	26/09/2026	Reestructuración formal del SRS conforme al formato EPIS y alineación con el MVP implementado.

El presente SRS mantiene el alcance funcional del proyecto y consolida los requerimientos a partir del Documento de Visión, el Informe de Factibilidad y la implementación actual del MVP. Los elementos de seguridad e infraestructura que aún corresponden a una fase de despliegue se identifican como pendientes o sujetos a validación.

ÍNDICE GENERAL

INTRODUCCIÓN.....	3
I. GENERALIDADES DEL PROYECTO.....	4
1. Nombre del proyecto.....	4
2. Visión.....	4
3. Misión.....	4
4. Organigrama.....	4
II. VISIONAMIENTO DEL PROYECTO.....	5
1. Descripción del problema.....	5
2. Objetivos de negocio.....	5
3. Objetivos de diseño.....	5
4. Alcance del proyecto.....	6
5. Viabilidad del sistema.....	6
6. Información obtenida del levantamiento de información.....	6
III. ANÁLISIS DE PROCESOS.....	7
a) Diagrama del Proceso Actual - Diagrama de actividades.....	7
b) Diagrama del Proceso Propuesto - Diagrama de actividades inicial.....	7
IV. ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE.....	8
a) Cuadro de Requerimientos Funcionales Inicial.....	8
b) Cuadro de Requerimientos No Funcionales.....	9
c) Cuadro de Requerimientos Funcionales Final.....	12
d) Reglas de Negocio.....	18
Parámetros de detección del MVP.....	19
V. FASE DE DESARROLLO.....	20
1. Perfiles de Usuario.....	20
2. Modelo Conceptual.....	20
a) Diagrama de Paquetes / Componentes.....	20
b) Diagrama de Casos de Uso.....	22
c) Escenarios de Caso de Uso (narrativa).....	22
CU-01: Gestionar sesión de usuario.....	22
CU-02: Recibir y analizar evento.....	24
CU-03: Consultar y gestionar alertas.....	25
CU-04: Analizar actividad de usuario.....	26
CU-05: Gestionar cuentas de analistas.....	27
CU-06: Consultar dashboard y eventos.....	29
3. Modelo Lógico.....	30
a) Análisis de Objetos.....	30
b) Diagrama de Actividades con Objetos.....	32

c) Diagrama de Secuencia.....	33
d) Diagrama de Clases / Modelo lógico.....	34
4. Matriz de trazabilidad y estado de implementación.....	34
Estado de requerimientos no funcionales relevantes para despliegue.....	35
5. Tecnologías relacionadas.....	36
CONCLUSIONES.....	37
RECOMENDACIONES.....	37
BIBLIOGRAFÍA.....	37
WEBGRAFÍA.....	38

INTRODUCCIÓN

El presente Documento de Especificación de Requerimientos de Software (SRS) define los requerimientos funcionales y no funcionales del Sistema web de detección y análisis de anomalías en operaciones de bases de datos NoSQL, 2026. El documento formaliza el comportamiento esperado del MVP, sus reglas de negocio, restricciones, perfiles de usuario y modelos de interacción, manteniendo coherencia con el Documento de Visión, el Informe de Factibilidad y la implementación disponible en el repositorio del proyecto.

El sistema recibe eventos asociados a operaciones READ, INSERT, UPDATE y DELETE, valida su estructura, los almacena en MongoDB y aplica reglas explicables para identificar operaciones masivas, alta frecuencia, fallos repetidos, actividad en horario inusual y desviaciones respecto del comportamiento histórico. Cuando corresponde, genera alertas con puntuación, severidad, motivos y evidencia. El panel web permite consultar eventos, alertas, estadísticas, análisis por usuario y fecha, y administrar cuentas de analistas.

El MVP utiliza Python 3.12, FastAPI, Pydantic, PyMongo asíncrono, MongoDB, HTML, CSS, JavaScript y Chart.js. La autenticación emplea JWT y contraseñas protegidas con Argon2. El sistema detecta e informa; no bloquea automáticamente operaciones sobre la base de datos y no sustituye los controles nativos de seguridad, un SIEM o una plataforma de respuesta automática a incidentes.

I. GENERALIDADES DEL PROYECTO

1. Nombre del proyecto

Sistema web de detección y análisis de anomalías en operaciones de bases de datos NoSQL, 2026.

2. Visión

Constituir una plataforma web clara, modular y extensible que centralice eventos de actividad NoSQL, detecte patrones que requieran revisión y proporcione a administradores y analistas evidencia comprensible para investigar comportamientos anómalos con mayor rapidez y trazabilidad.

3. Misión

Apoyar la supervisión de operaciones NoSQL mediante un flujo integrado de recepción, validación, análisis, almacenamiento y visualización de eventos, utilizando reglas transparentes y análisis histórico sin sustituir la decisión humana ni los controles nativos de seguridad de la base de datos.

4. Organigrama

El proyecto es desarrollado por un equipo académico de dos integrantes. La distribución de responsabilidades permite organizar el trabajo sin impedir la participación conjunta en arquitectura, integración, validación, pruebas y documentación.

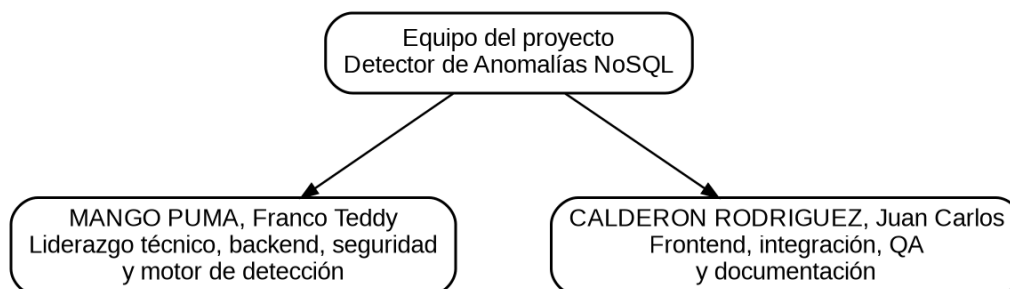


Figura 1. Organización funcional del equipo del proyecto.

II. VISIONAMIENTO DEL PROYECTO

1. Descripción del problema

En entornos que utilizan bases de datos NoSQL, la actividad puede quedar distribuida entre registros técnicos y consultas aisladas. Cuando la revisión depende de inspección manual, resulta difícil reconocer con rapidez eliminaciones o actualizaciones masivas, ráfagas de operaciones, fallos reiterados, actividad fuera del horario esperado o cambios relevantes en el comportamiento de un usuario.

El problema central es la ausencia de un flujo integrado que normalice eventos, aplique criterios de detección, conserve evidencia, clasifique severidad y permita comparar el comportamiento actual con el histórico. La propuesta responde a esta necesidad mediante una API, un motor de reglas explicable, persistencia en MongoDB y una interfaz web orientada al análisis.

2. Objetivos de negocio

- Centralizar los eventos relevantes de operaciones NoSQL en un repositorio consultable y trazable.
- Reducir el tiempo necesario para identificar patrones que requieren revisión humana.
- Priorizar anomalías mediante puntuación y severidad, conservando los motivos que originan cada alerta.
- Facilitar el análisis individual de un usuario en una fecha concreta y su comparación con días anteriores.
- Separar funciones administrativas y de análisis mediante roles de acceso.

3. Objetivos de diseño

- Mantener una interfaz web clara y responsive con navegación consistente.

- Separar frontend, API, autenticación, detección, servicios y acceso a datos para favorecer mantenibilidad.
- Utilizar reglas de detección interpretables y configurables desde código en el MVP.
- Evitar que la visualización dependa directamente de MongoDB; toda interacción del frontend se realiza mediante la API.
- Mantener tiempos de respuesta adecuados para las consultas habituales del MVP.

4. Alcance del proyecto

El alcance comprende una API FastAPI, persistencia MongoDB, motor de detección basado en reglas y estadística simple, simulador de eventos y una interfaz web con autenticación. Los módulos principales son Login, Dashboard, Eventos, Alertas, Estadísticas, Análisis de Actividad y Gestión de Usuarios para el Administrador.

Quedan fuera del alcance de esta versión el bloqueo automático de operaciones, la sustitución de herramientas SIEM/EDR, la integración directa con una base empresarial real, el uso de aprendizaje automático avanzado, las notificaciones externas y la respuesta automática a incidentes.

5. Viabilidad del sistema

La viabilidad técnica, económica y operativa fue evaluada favorablemente para un MVP académico y un piloto de bajo volumen. El stack se basa en tecnologías ampliamente disponibles y sin costos obligatorios de licencia para el entorno de desarrollo. La precisión del detector depende de la calidad de los eventos y de la calibración de los umbrales en cada entorno real.

6. Información obtenida del levantamiento de información

Los requerimientos se obtuvieron mediante análisis del problema, revisión iterativa del prototipo, definición de escenarios controlados del simulador, contraste con la estructura del repositorio y validación funcional durante el desarrollo. No se utilizaron datos reales de una organización ni se asumieron entrevistas con un cliente externo. Los escenarios cubren operación normal, eliminación o actualización masiva, fallos repetidos, alta frecuencia, horario inusual y desviación de comportamiento.

III. ANÁLISIS DE PROCESOS

a) Diagrama del Proceso Actual - Diagrama de actividades

Como situación de referencia se considera un entorno donde las operaciones NoSQL generan registros técnicos que deben ser revisados manualmente. La dispersión de información y la ausencia de una clasificación automática incrementan el esfuerzo necesario para identificar cambios relevantes y reducen la trazabilidad de la investigación.

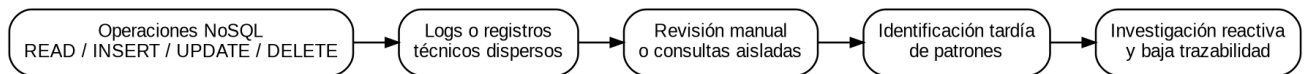


Figura 2. Proceso de referencia sin el sistema propuesto.

b) Diagrama del Proceso Propuesto - Diagrama de actividades inicial

El proceso propuesto recibe un evento desde el simulador o una fuente autorizada, valida el esquema, persiste el evento, ejecuta el motor de detección, genera o agrupa una alerta cuando existe anomalía y expone los resultados mediante la API para su consulta desde la interfaz web.

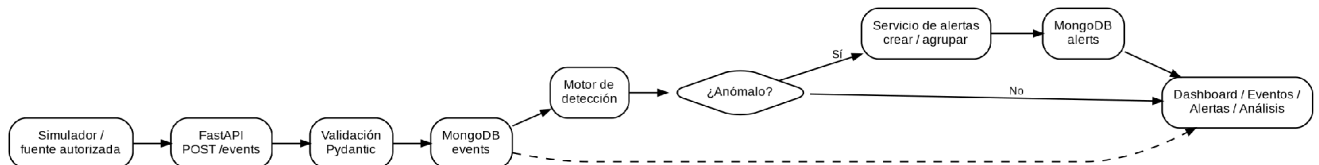


Figura 3. Flujo propuesto de recepción, análisis y visualización de eventos.

IV. ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE

Los requerimientos se presentan en tres niveles: una consolidación funcional inicial, los requerimientos no funcionales de calidad y operación, y el catálogo funcional final utilizado como referencia para la implementación y la trazabilidad. La redacción emplea la forma “El sistema deberá...” para expresar comportamiento verificable.

a) Cuadro de Requerimientos Funcionales Inicial

ID	Nombre	Descripción	Actor	Prioridad
RFI-001	Autenticación y roles	Permitir acceso mediante usuario y contraseña, diferenciando Administrador y Analista.	Administrador / Analista	Alta
RFI-002	Recepción de eventos	Recibir eventos NoSQL desde un simulador o fuente autorizada y validar su estructura.	Fuente / Sistema	Alta
RFI-003	Persistencia de eventos	Registrar eventos con usuario, operación, colección, registros afectados, resultado y fecha/hora.	Sistema	Alta
RFI-004	Detección de anomalías	Analizar operaciones masivas, alta frecuencia, fallos repetidos, horario inusual y desviaciones históricas.	Sistema	Alta
RFI-005	Gestión de alertas	Generar alertas con score, severidad, razones y evidencia; permitir	Sistema / Analista	Alta

ID	Nombre	Descripción	Actor	Prioridad
		seguimiento por estado.		
RFI-006	Monitoreo web	Mostrar dashboard y permitir consultar eventos y alertas con filtros.	Administrador / Analista	Alta
RFI-007	Análisis de actividad	Analizar un usuario y fecha, mostrar actividad por hora y comparar con días anteriores.	Administrador / Analista	Alta
RFI-008	Gestión de usuarios	Permitir al Administrador crear, activar/desactivar y restablecer contraseñas de analistas.	Administrador	Alta

b) Cuadro de Requerimientos No Funcionales

ID	Categoría	Descripción	Prioridad
RNF-001	Seguridad	Las contraseñas deberán almacenarse mediante hash seguro basado en Argon2, nunca en texto plano.	Alta
RNF-002	Seguridad	El acceso a módulos internos deberá protegerse mediante JWT y autorización según roles Administrador y Analista.	Alta
RNF-003	Seguridad	Antes de un despliegue real, la recepción de eventos externos deberá	Alta

ID	Categoría	Descripción	Prioridad
		protegerse mediante una credencial de servicio, API Key o control equivalente.	
RNF-004	Seguridad	Claves secretas, credenciales y parámetros sensibles deberán administrarse mediante variables de entorno y quedar fuera del repositorio.	Alta
RNF-005	Seguridad	En producción, la comunicación entre navegador, API y servicios deberá utilizar HTTPS/TLS.	Alta
RNF-006	Rendimiento	Las consultas habituales de Dashboard, Eventos, Alertas y Análisis deberán responder en un tiempo objetivo menor a 3 segundos bajo condiciones normales del MVP.	Alta
RNF-007	Rendimiento	El análisis de anomalías deberá ejecutarse automáticamente después de recibir un evento, sin intervención manual.	Alta
RNF-008	Integridad	Los datos recibidos deberán ser validados antes de almacenarse, rechazando valores incompletos o inválidos.	Alta

ID	Categoría	Descripción	Prioridad
RNF-009	Confiabilidad	El sistema deberá mantener consistencia y trazabilidad entre eventos, resultados de análisis y alertas mediante identificadores y marcas de tiempo.	Alta
RNF-010	Usabilidad	La interfaz deberá presentar información comprensible mediante indicadores, tablas, gráficos, severidades y mensajes claros.	Alta
RNF-011	Compatibilidad	La aplicación deberá funcionar en navegadores modernos como Chrome, Edge y Firefox.	Media
RNF-012	Adaptabilidad	La interfaz deberá ser responsive y visualizarse adecuadamente en escritorio, laptop, tablet y móvil.	Media
RNF-013	Mantenibilidad	La arquitectura deberá separar autenticación, acceso a datos, detección, servicios, rutas y frontend para facilitar cambios.	Alta
RNF-014	Escalabilidad	La solución deberá permitir incrementar eventos, usuarios monitoreados y alertas sin reestructurar completamente la aplicación.	Media

ID	Categoría	Descripción	Prioridad
RNF-015	Portabilidad	El sistema deberá poder instalarse en distintos entornos compatibles con Python, FastAPI y MongoDB mediante configuración externa.	Media

c) Cuadro de Requerimientos Funcionales Final

ID	Nombre	Descripción	Actor	Prioridad
RF-001	Gestionar inicio de sesión	El sistema deberá permitir el inicio de sesión de usuarios autorizados mediante nombre de usuario y contraseña.	Administrador / Analista	Alta
RF-002	Autenticar y autorizar	El sistema deberá gestionar la autenticación mediante tokens JWT y restringir el acceso a los módulos protegidos según el rol del usuario.	Sistema	Alta
RF-003	Gestionar roles	El sistema deberá diferenciar los permisos correspondientes a los roles Administrador y Analista.	Sistema	Alta
RF-004	Cerrar sesión	El sistema deberá permitir cerrar la sesión y eliminar las credenciales de autenticación	Administrador / Analista	Alta

ID	Nombre	Descripción	Actor	Prioridad
		almacenadas en el navegador.		
RF-005	Recibir eventos	El sistema deberá recibir operaciones o eventos provenientes del simulador o fuente autorizada mediante la API y validar su estructura antes de procesarlos.	Fuente / Sistema	Alta
RF-006	Registrar eventos	El sistema deberá registrar en MongoDB cada evento recibido, incluyendo usuario, operación, colección afectada, registros involucrados, resultado y fecha/hora.	Sistema	Alta
RF-007	Analizar eventos	El sistema deberá analizar automáticamente cada evento registrado para determinar si corresponde a un comportamiento normal o anómalo.	Sistema	Alta
RF-008	Detectar operaciones masivas	El sistema deberá detectar operaciones masivas DELETE y UPDATE de acuerdo con los	Sistema	Alta

ID	Nombre	Descripción	Actor	Prioridad
		umbrales establecidos.		
RF-009	Detectar alta frecuencia	El sistema deberá detectar comportamientos de alta frecuencia cuando un usuario realice una cantidad elevada de operaciones dentro de un período determinado.	Sistema	Alta
RF-010	Detectar fallos repetidos	El sistema deberá detectar fallos repetidos cuando un usuario acumule múltiples operaciones no exitosas dentro de un intervalo definido.	Sistema	Alta
RF-011	Detectar horario inusual	El sistema deberá detectar operaciones realizadas en horarios considerados inusuales según la configuración del sistema.	Sistema	Alta
RF-012	Detectar desviación de comportamiento	El sistema deberá detectar desviaciones significativas respecto al comportamiento histórico de un usuario considerando sus	Sistema	Alta

ID	Nombre	Descripción	Actor	Prioridad
		operaciones anteriores.		
RF-013	Calcular score y severidad	El sistema deberá calcular una puntuación de anomalía y determinar su nivel de severidad como low, medium, high o critical.	Sistema	Alta
RF-014	Generar alertas	El sistema deberá generar y almacenar alertas cuando se detecte una anomalía, incluyendo tipo de detección, motivo, evidencia, puntuación y severidad.	Sistema	Alta
RF-015	Consultar alertas	El sistema deberá permitir consultar, filtrar y visualizar el detalle de las alertas generadas, incluyendo su evidencia y número de ocurrencias.	Administrador / Analista	Alta
RF-016	Gestionar estado de alertas	El sistema deberá permitir cambiar el estado de una alerta entre Pendiente, Revisada y Resuelta.	Administrador / Analista	Alta
RF-017	Visualizar dashboard	El sistema deberá mostrar un Dashboard con	Administrador / Analista	Alta

ID	Nombre	Descripción	Actor	Prioridad
		indicadores generales de eventos procesados, anomalías, alertas pendientes y alertas críticas.		
RF-018	Consultar eventos	El sistema deberá permitir consultar el historial de eventos y filtrarlo por usuario, tipo de operación, resultado y condición normal o anómala.	Administrador / Analista	Alta
RF-019	Seleccionar usuario y fecha	El sistema deberá permitir seleccionar un usuario monitoreado y una fecha específica para analizar la actividad realizada durante dicho día.	Administrador / Analista	Alta
RF-020	Resumir actividad diaria	El sistema deberá mostrar total de eventos, operaciones exitosas y fallidas, anomalías, registros afectados y cantidades READ, INSERT, UPDATE y DELETE del usuario seleccionado.	Administrador / Analista	Alta

ID	Nombre	Descripción	Actor	Prioridad
RF-021	Visualizar actividad por hora e historial	El sistema deberá mostrar la distribución de actividad por hora y el historial completo de operaciones del día seleccionado.	Administrador / Analista	Media
RF-022	Comparar comportamiento histórico	El sistema deberá comparar la actividad del usuario seleccionado con días anteriores, calculando promedios y variaciones porcentuales por total y tipo de operación.	Administrador / Analista	Alta
RF-023	Identificar cambios y filtrar análisis	El sistema deberá presentar cambios relevantes en el comportamiento y permitir filtrar el historial entre eventos normales y anómalos.	Administrador / Analista	Alta
RF-024	Gestionar cuentas de analistas	El sistema deberá permitir al Administrador consultar las cuentas del sistema y crear nuevas cuentas con rol Analista.	Administrador	Alta
RF-025	Administrar acceso de analistas	El sistema deberá permitir al Administrador activar o desactivar cuentas	Administrador / Sistema	Alta

ID	Nombre	Descripción	Actor	Prioridad
		de analistas y restablecer sus contraseñas, protegiendo la cuenta principal del Administrador.		

d) Reglas de Negocio

Regla	Descripción
RN-01	El sistema detecta e informa anomalías; no bloquea automáticamente las operaciones realizadas sobre la base de datos.
RN-02	Una operación puede activar una o más reglas de detección simultáneamente; el score total se limita a 100 puntos.
RN-03	Los usuarios Administrador y Analista son cuentas de acceso a la plataforma; usuario01, usuario02 u otras identidades monitoreadas no necesariamente son cuentas del panel.
RN-04	La gestión de cuentas del sistema es exclusiva del Administrador; el Analista no puede acceder a dichas funciones aunque conozca el endpoint.
RN-05	La cuenta principal del Administrador no puede ser desactivada desde el módulo de gestión de usuarios y su contraseña no se restablece desde ese módulo.
RN-06	Las alertas similares de un mismo usuario pueden agruparse dentro de una ventana corta, incrementando el contador de ocurrencias en lugar de crear registros redundantes.

Parámetros de detección del MVP

Parámetro	Valor / criterio actual
Operación masiva DELETE	Advertencia desde 200 registros; condición crítica desde 1000 registros afectados. Puntajes actuales: 50 y 75 respectivamente.
Operación masiva UPDATE	Advertencia desde 500 registros; umbral superior desde 2000 registros afectados. Puntajes actuales: 40 y 60 respectivamente.
Alta frecuencia	Ventana de 60 segundos; umbral base de 100 eventos por usuario. Con 200 o más operaciones el puntaje aumenta.
Fallos repetidos	Ventana de 5 minutos; umbral base de 5 operaciones fallidas por usuario. Con 10 o más fallos el puntaje aumenta.
Horario inusual	Entre 00:00 y 05:59 en la zona horaria de negocio configurada (UTC-5 en el MVP).
Desviación histórica	Mínimo 20 muestras previas; z-score ≥ 3.0 cuando existe desviación estándar utilizable. Puntaje actual 30.
Agrupación de alertas	Ventana de 5 minutos para alertas pendientes con la misma firma y usuario.
Severidad por score	Low: 1-24; Medium: 25-49; High: 50-74; Critical: 75-100.

Los umbrales corresponden al MVP y deben recalibrarse con datos representativos antes de una implantación real. Una anomalía indica un comportamiento que requiere revisión y no equivale, por sí sola, a un incidente de seguridad.

V. FASE DE DESARROLLO

1. Perfiles de Usuario

Perfil	Función	Necesidades / permisos
Administrador	Configura y administra el acceso a la plataforma.	Consultar dashboard, eventos, alertas y análisis; cambiar estados de alertas; crear, activar/desactivar y restablecer contraseñas de analistas.
Analista	Investiga eventos y alertas generadas por el detector.	Consultar dashboard, eventos y alertas; revisar evidencia; cambiar estado de alertas; analizar actividad individual e histórica.
Usuario monitoreado	Identidad asociada a eventos de actividad NoSQL. No es necesariamente una cuenta de acceso web.	Sus operaciones se registran y pueden ser analizadas por el motor de detección.
Simulador / Fuente de eventos	Componente externo que envía eventos al backend para pruebas o integración.	Enviar eventos con el esquema definido y recibir el resultado del análisis.

2. Modelo Conceptual

a) Diagrama de Paquetes / Componentes

La implementación actual utiliza una organización modular. FastAPI registra rutas separadas para autenticación, usuarios, eventos, alertas, dashboard, estadísticas y análisis. La seguridad se concentra en security.py; la detección en engine.py, rules.py y config.py; el servicio de alertas en alert_service.py; y el acceso a MongoDB en connection.py. El frontend consume únicamente la API.

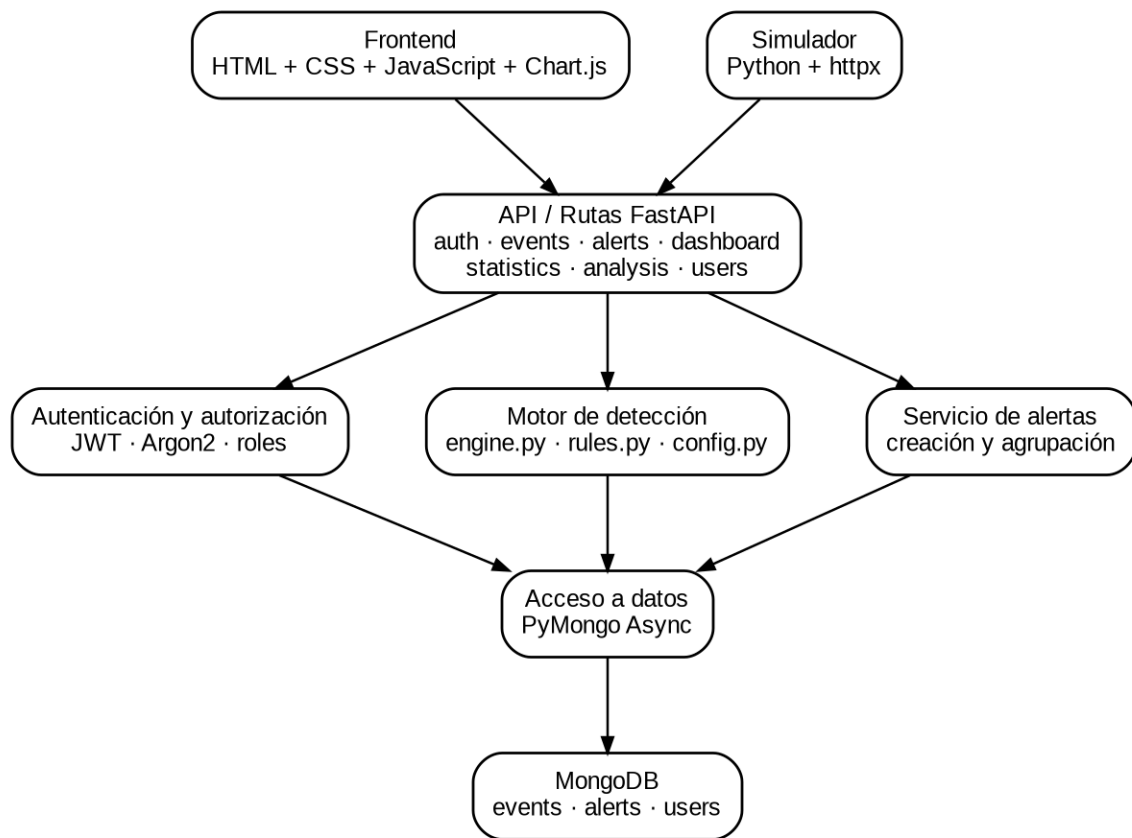


Figura 4. Diagrama de paquetes y componentes principales del sistema.

b) Diagrama de Casos de Uso

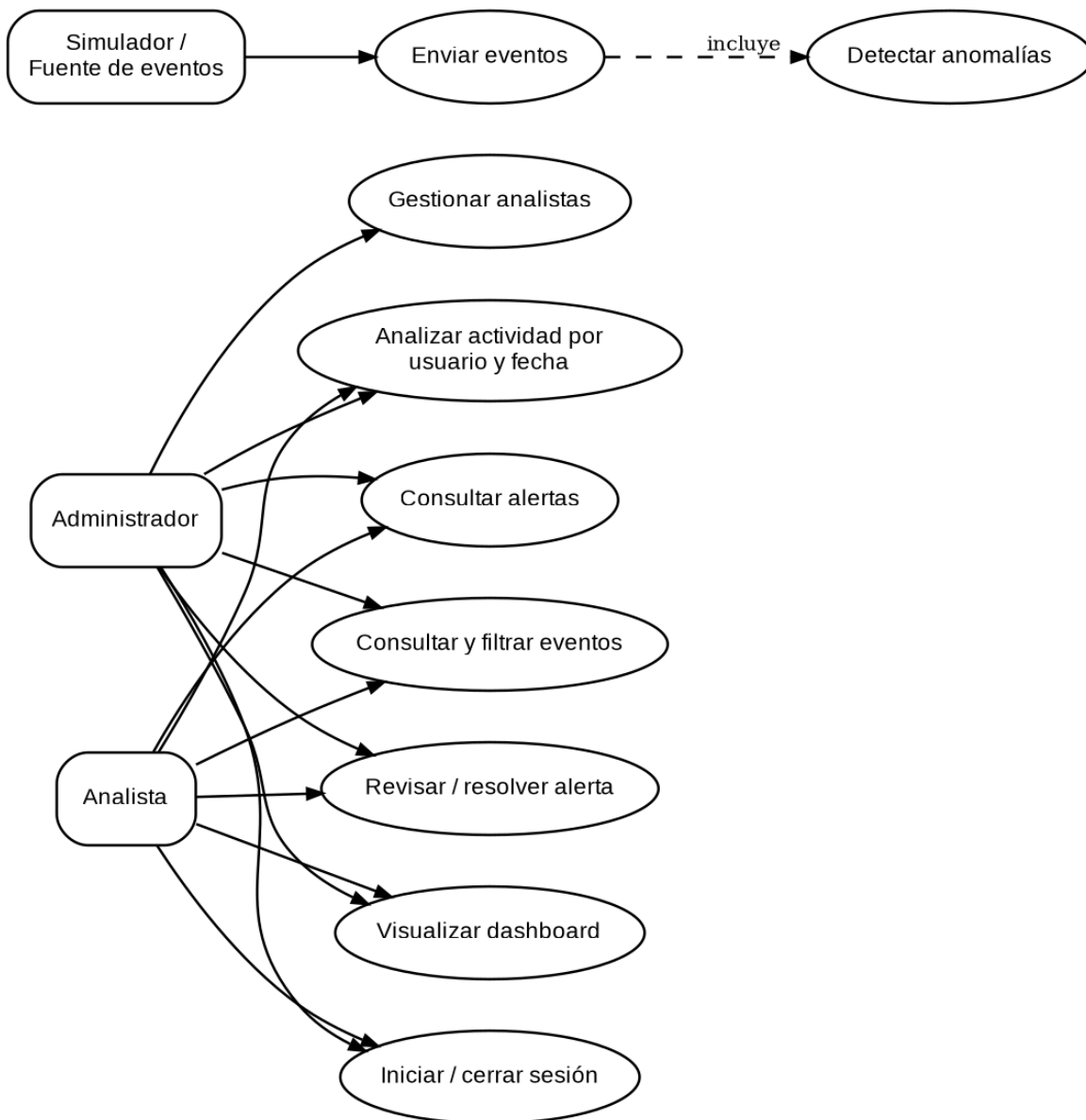


Figura 5. Casos de uso principales y actores del sistema.

c) Escenarios de Caso de Uso (narrativa)

Se documentan los escenarios que concentran los flujos principales del sistema. Los requisitos detallados se encuentran en el catálogo funcional y se relacionan posteriormente en la matriz de trazabilidad.

CU-01: Gestionar sesión de usuario

Campo	Descripción
-------	-------------

Id Caso de Uso	CU-01
Nombre	Gestionar sesión de usuario
Tipo	Obligatorio
Requisito ID (RF)	RF-001 a RF-004
Versión	1.0
Autor	Equipo de desarrollo
Actores	Administrador, Analista
Descripción	Permitir iniciar y cerrar sesión utilizando credenciales válidas, aplicando autenticación JWT y permisos por rol.
Precondiciones	1. La cuenta debe existir y estar activa. 2. La API y MongoDB deben estar disponibles.
Postcondiciones	1. El usuario accede a los módulos permitidos por su rol o finaliza su sesión.

Flujo normal de eventos

Paso	Actor	Sistema
1	Accede a la página de login e ingresa usuario y contraseña.	Valida la cuenta y la contraseña protegida con Argon2.
2	Envía el formulario.	Si las credenciales son válidas, genera el token JWT y devuelve el rol.
3	Navega por el sistema.	Autoriza cada endpoint protegido según el token y el rol.
4	Selecciona Cerrar sesión.	El frontend elimina la credencial local y redirige al login.

Flujos de excepción

Código	Descripción
E001	Credenciales incorrectas o cuenta inactiva: acceso no autorizado.

Código	Descripción
E002	Token inválido o vencido: se deniega el recurso protegido.

CU-02: Recibir y analizar evento

Campo	Descripción
Id Caso de Uso	CU-02
Nombre	Recibir y analizar evento
Tipo	Obligatorio
Requisito ID (RF)	RF-005 a RF-014
Versión	1.0
Autor	Equipo de desarrollo
Actores	Simulador / Fuente de eventos, Sistema
Descripción	Recibir un evento, validarlo, almacenarlo, aplicar reglas de detección y generar una alerta cuando corresponda.
Precondiciones	1. La API está disponible. 2. El evento cumple el contrato esperado o puede ser validado.
Postcondiciones	1. El evento queda persistido con resultado de análisis; si es anómalo, queda asociado a una alerta.

Flujo normal de eventos

Paso	Actor	Sistema
1	Envía un evento con user_id, operación, colección, registros afectados, resultado y timestamp.	Valida tipos, valores permitidos y obligatoriedad mediante Pydantic.
2	Espera la respuesta del servicio.	Inserta el evento en MongoDB y ejecuta las reglas de detección.

Paso	Actor	Sistema
3	-	Calcula score y severidad; si existe anomalía, crea o agrupa una alerta.
4	Recibe la respuesta.	Devuelve el evento procesado y el resumen del análisis.

Flujos de excepción

Código	Descripción
E001	Evento con estructura inválida: se rechaza con error de validación.
E002	Fuente no autorizada en despliegue con credencial de servicio: se rechaza la petición.
E003	Error de base de datos: se informa fallo del servicio y no se confirma persistencia.

CU-03: Consultar y gestionar alertas

Campo	Descripción
Id Caso de Uso	CU-03
Nombre	Consultar y gestionar alertas
Tipo	Obligatorio
Requisito ID (RF)	RF-015 y RF-016
Versión	1.0
Autor	Equipo de desarrollo
Actores	Administrador, Analista
Descripción	Consultar alertas, aplicar filtros, abrir su detalle y cambiar su estado de seguimiento.
Precondiciones	1. El usuario inició sesión. 2. Existen alertas registradas o la consulta puede devolver lista vacía.

Postcondiciones	1. La alerta puede quedar Pendiente, Revisada o Resuelta según la acción realizada.
-----------------	---

Flujo normal de eventos

Paso	Actor	Sistema
1	Accede al módulo Alertas.	Lista alertas ordenadas y aplica filtros por severidad, estado, usuario u operación.
2	Selecciona Ver detalle.	Devuelve motivos, detecciones, evidencia, score, severidad y ocurrencias.
3	Cambia el estado de la alerta.	Valida el estado solicitado y actualiza updated_at.

Flujos de excepción

Código	Descripción
E001	Alerta inexistente: se responde 404.
E002	Token inválido o sesión no autorizada: se responde 401/403.

CU-04: Analizar actividad de usuario

Campo	Descripción
Id Caso de Uso	CU-04
Nombre	Analizar actividad de usuario
Tipo	Obligatorio
Requisito ID (RF)	RF-019 a RF-023
Versión	1.0
Autor	Equipo de desarrollo
Actores	Administrador, Analista

Descripción	Analizar la actividad de un usuario monitoreado para una fecha y compararla con días anteriores.
Precondiciones	1. El usuario de plataforma está autenticado. 2. Existen eventos del usuario o la consulta puede devolver un día sin actividad.
Postcondiciones	1. Se muestran resumen, operaciones, actividad por hora, comparación histórica, cambios relevantes e historial filtrable.

Flujo normal de eventos

Paso	Actor	Sistema
1	Selecciona usuario, fecha y período histórico.	Obtiene eventos del día dentro de la zona horaria de negocio.
2	Presiona Analizar.	Calcula totales, éxitos, fallos, anomalías y registros afectados.
3	Revisa gráficos y comparación.	Calcula promedios históricos y variaciones porcentuales.
4	Filtra el historial por análisis.	Muestra todos, normales o anómalos sin alterar los datos originales.

Flujos de excepción

Código	Descripción
E001	Sin eventos en la fecha: se muestran contadores en cero y tabla vacía.
E002	Sin base histórica suficiente: la variación se presenta como no disponible cuando no puede calcularse válidamente.

CU-05: Gestionar cuentas de analistas

Campo	Descripción
Id Caso de Uso	CU-05

Nombre	Gestionar cuentas de analistas
Tipo	Obligatorio
Requisito ID (RF)	RF-024 y RF-025
Versión	1.0
Autor	Equipo de desarrollo
Actores	Administrador
Descripción	Permitir al Administrador crear cuentas de analistas, activar/desactivar su acceso y restablecer contraseñas.
Precondiciones	1. El Administrador inició sesión. 2. La cuenta principal de Administrador existe y está activa.
Postcondiciones	1. Las cuentas de analistas quedan creadas o actualizadas sin exponer el hash de contraseña.

Flujo normal de eventos

Paso	Actor	Sistema
1	Accede a Usuarios.	Lista cuentas sin devolver password_hash.
2	Selecciona Crear analista e ingresa usuario y contraseña.	Valida unicidad, genera hash Argon2 y crea la cuenta activa.
3	Selecciona Activar o Desactivar.	Actualiza el campo active; impide desactivar al Administrador principal.
4	Selecciona Contraseña.	Recalcula el hash y actualiza la credencial del analista.

Flujos de excepción

Código	Descripción
E001	Nombre de usuario duplicado: se rechaza con conflicto.

Código	Descripción
E002	Un Analista intenta usar endpoints administrativos: se responde 403.
E003	Intento de desactivar al Administrador principal: operación rechazada.

CU-06: Consultar dashboard y eventos

Campo	Descripción
Id Caso de Uso	CU-06
Nombre	Consultar dashboard y eventos
Tipo	Obligatorio
Requisito ID (RF)	RF-017 y RF-018
Versión	1.0
Autor	Equipo de desarrollo
Actores	Administrador, Analista
Descripción	Consultar el estado general del sistema y explorar los eventos registrados mediante filtros.
Precondiciones	1. El usuario inició sesión.
Postcondiciones	1. Se presenta un resumen general y/o un conjunto de eventos según criterios seleccionados.

Flujo normal de eventos

Paso	Actor	Sistema
1	Accede al Dashboard.	Calcula totales de eventos, anomalías y alertas por estado/severidad.
2	Accede a Eventos.	Lista eventos recientes.
3	Aplica filtros.	Consulta por usuario, operación, resultado y condición anómala.

Paso	Actor	Sistema
4	Selecciona un evento o revisa la tabla.	Muestra los datos persistidos y su clasificación de análisis.

Flujos de excepción

Código	Descripción
E001	No existen datos para los filtros: se devuelve una lista vacía con mensaje apropiado.
E002	Error de conexión con MongoDB: la API informa fallo del servicio.

3. Modelo Lógico

a) Análisis de Objetos

El sistema utiliza MongoDB y no implementa un ORM relacional. Por ello, el análisis de objetos representa las entidades lógicas persistidas y los objetos de validación/intercambio utilizados por el backend.

Objeto	Propósito	Atributos principales	Tratamiento
Event	Evento de actividad NoSQL recibido por la API.	event_id, user_id, operation, collection, records_affected, success, timestamp, analyzed, is_anomalous, anomaly_score, severity, alert_id	Se crea desde EventCreate, se almacena en events y luego se actualiza con el resultado del análisis.
Alert	Alerta generada cuando un evento activa una o más reglas.	alert_id, event_id/latest_event_id, user_id, operation, types, score, severity, reasons, detections, status, occurrence_count, created_at, updated_at	Se crea o agrupa mediante alert_service y se almacena en alerts.
User	Cuenta de acceso al panel web.	username, password_hash, role, active, created_at,	Se autentica mediante JWT; el Administrador

Objeto	Propósito	Atributos principales	Tratamiento
		created_by, updated_at, updated_by	gestiona cuentas con rol analyst.
Detection Result	Objeto temporal del motor de detección.	is_anomalous, score, severity, detections	No constituye una colección independiente; su resultado se integra en Event y Alert.

b) Diagrama de Actividades con Objetos

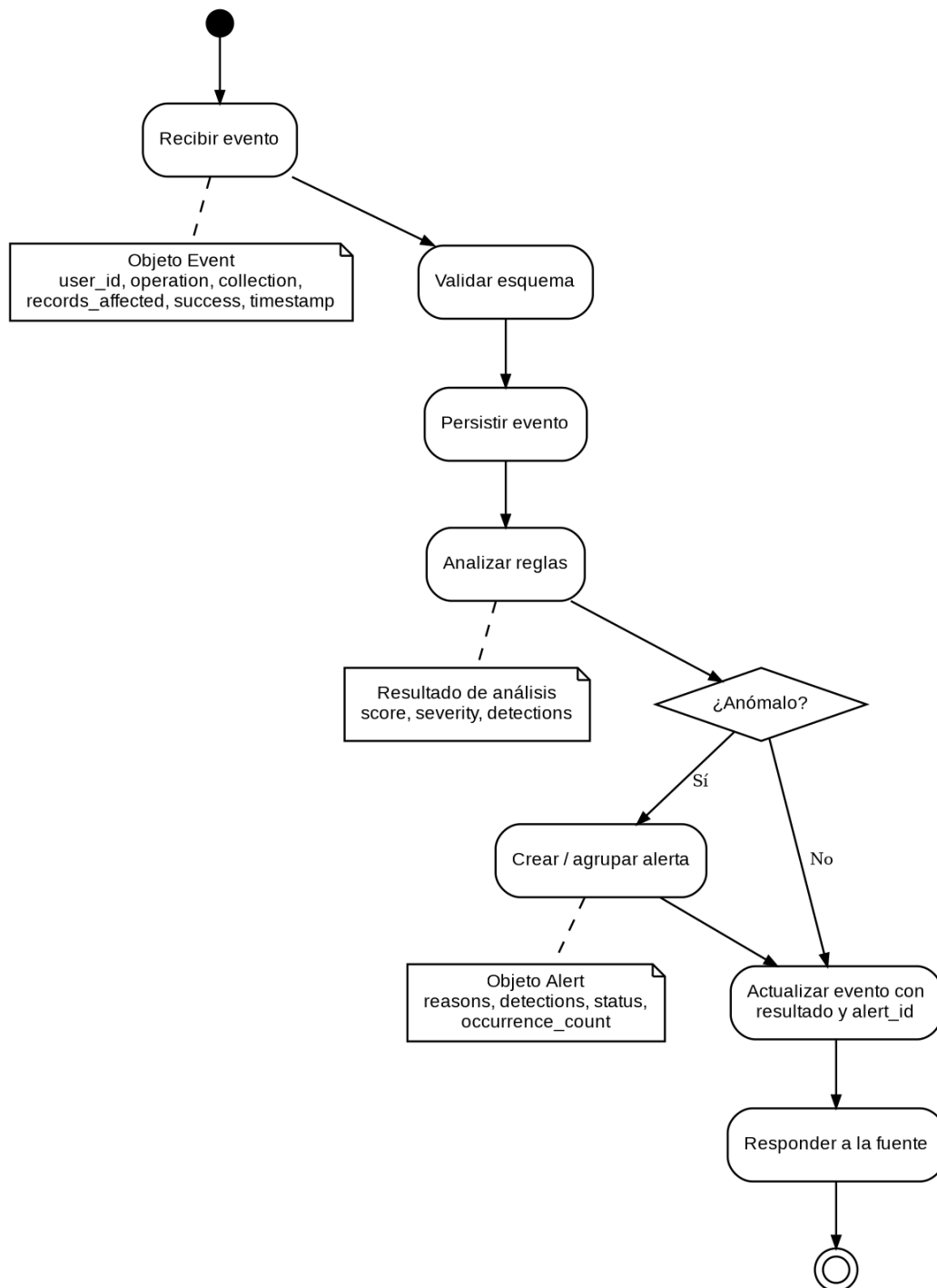
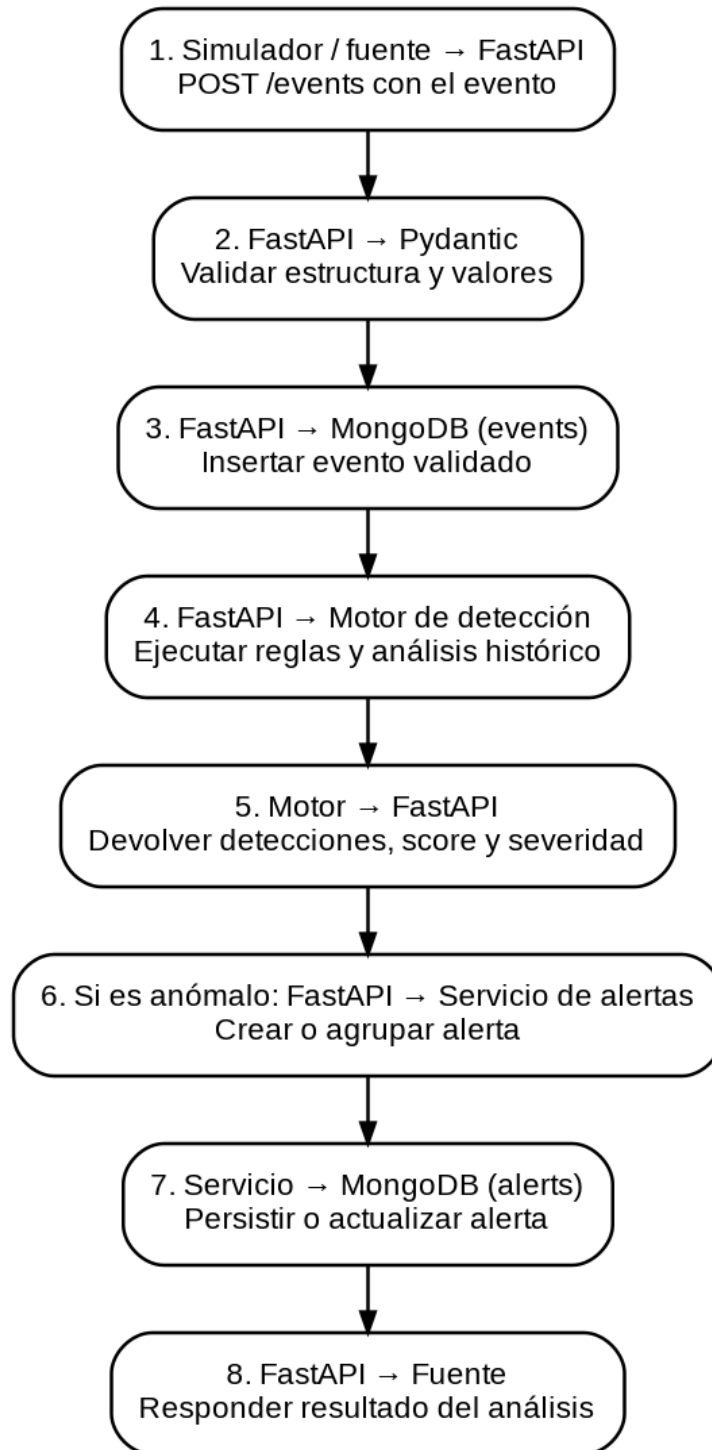


Figura 6. Diagrama de actividades con objetos para el procesamiento de un evento.

c) Diagrama de Secuencia

*Figura 7. Secuencia simplificada de recepción, análisis y generación de alertas.*

d) Diagrama de Clases / Modelo lógico

El siguiente diagrama representa las entidades lógicas principales. En la implementación MongoDB estas estructuras se materializan como documentos en las colecciones events, alerts y users, por lo que el diagrama debe entenderse como un modelo lógico y no como un conjunto de clases ORM.

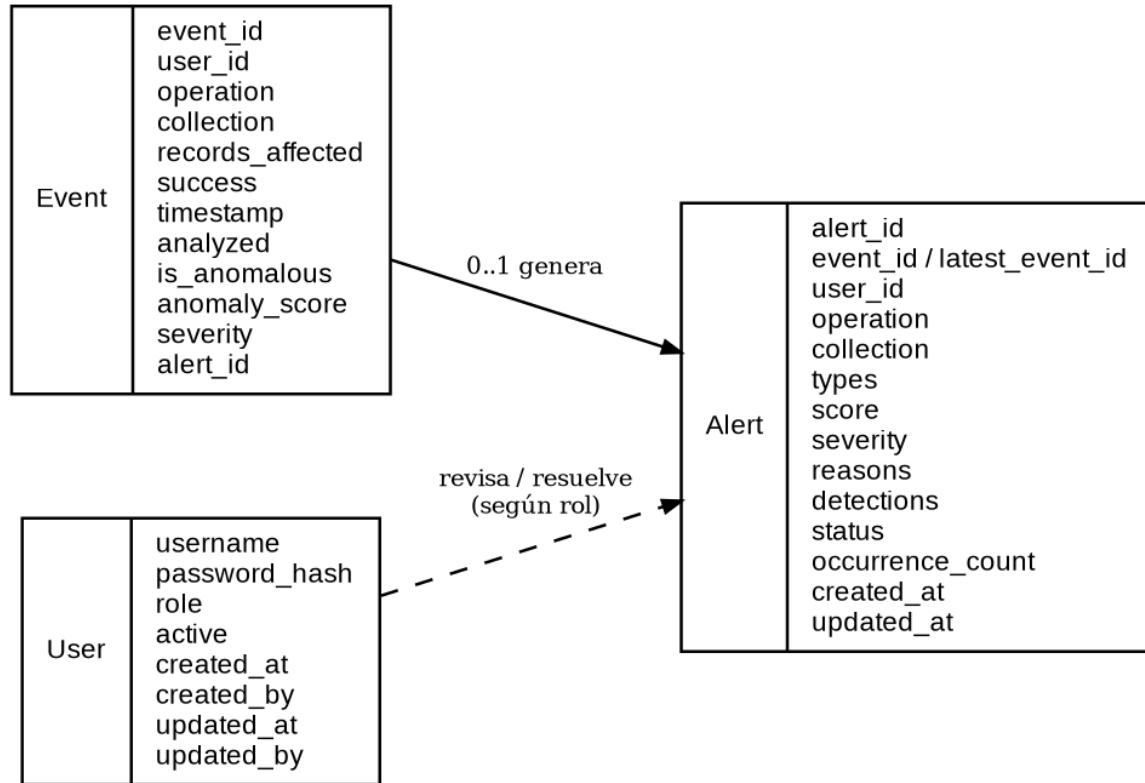


Figura 8. Modelo lógico simplificado de Event, Alert y User.

4. Matriz de trazabilidad y estado de implementación

Módulo	Requisitos	Componentes relacionados	Estado
Autenticación y roles	RF-001 a RF-004	auth/security.py, routes/auth.py, auth.js, login.js	Implementado
Recepción de eventos	RF-005 y RF-006	routes/events.py, schemas/event.py, simulator.py	Implementado
Motor de detección	RF-007 a RF-014	detection/rules.py, detection/engine.py,	Implementado

Módulo	Requisitos	Componentes relacionados	Estado
		detection/config.py, services/alert_service.py	
Alertas	RF-015 y RF-016	routes/alerts.py, alerts.html, alerts.js	Implementado
Dashboard y eventos	RF-017 y RF-018	routes/dashboard.py, routes/statistics.py, routes/events.py, frontend	Implementado
Análisis de actividad	RF-019 a RF-023	routes/analysis.py, activity-analysis.html/js	Implementado
Gestión de usuarios	RF-024 y RF-025	routes/users.py, users.html/js	Implementado

Estado de requerimientos no funcionales relevantes para despliegue

RNF	Evidencia / observación	Estado
RNF-001 / RNF-002	Hash Argon2, JWT y control de roles en backend.	Implementado
RNF-003	El endpoint POST /events aún no incorpora credencial de servicio/API Key en la implementación revisada.	Pendiente antes de despliegue externo
RNF-004	La configuración sensible se obtiene mediante variables de entorno (.env).	Implementado; verificar secretos del entorno
RNF-005	HTTPS depende de la infraestructura o proxy de despliegue.	Pendiente de infraestructura
RNF-006	Existe objetivo < 3 s, pero no se evidencia una prueba de carga formal.	Por validar
RNF-013	La estructura del backend separa rutas, seguridad, detección,	Implementado

RNF	Evidencia / observación	Estado
	servicios, configuración y acceso a datos.	
RNF-014	La arquitectura admite crecimiento, pero no existe validación formal de escalabilidad/carga.	Por validar
RNF-015	Configuración externa y stack portable en entornos Python/FastAPI/MongoDB.	Implementado a nivel de diseño

5. Tecnologías relacionadas

Área	Tecnologías del MVP
Backend	Python 3.12, FastAPI, Pydantic
Persistencia	MongoDB, PyMongo AsyncMongoClient
Seguridad	JWT (PyJWT), Argon2 mediante pwplib, variables de entorno
Frontend	HTML5, CSS3, JavaScript, Chart.js
Simulación / integración	Python, httpx, python-dotenv
Desarrollo	Git / GitHub, entorno virtual, Swagger/OpenAPI generado por FastAPI

CONCLUSIONES

- El SRS consolida 25 requerimientos funcionales y 15 no funcionales coherentes con el alcance del Documento de Visión, el Informe de Factibilidad y la implementación del MVP.
- El núcleo funcional se concentra en la recepción de eventos, detección explicable de anomalías, generación y seguimiento de alertas, análisis histórico y control de acceso por roles.
- La arquitectura mantiene separación entre usuarios de la plataforma (Administrador y Analista) y las identidades monitoreadas presentes en los eventos.
- La implementación revisada cubre el flujo funcional principal y las rutas de consulta/administración; los controles propios de un despliegue productivo, como HTTPS y protección de la ingesta externa, deben completarse antes de exposición pública.
- El sistema debe entenderse como una herramienta de apoyo a la revisión humana: una detección anómala no implica automáticamente un incidente y el sistema no bloquea operaciones de base de datos.

RECOMENDACIONES

- Proteger POST /events con una credencial de servicio, API Key, lista de red u otro mecanismo equivalente antes de desplegarlo fuera de un entorno controlado.
- Mantener SECRET_KEY, credenciales y cadenas de conexión fuera del repositorio y habilitar HTTPS/TLS en el entorno publicado.
- Incorporar pruebas automatizadas para autenticación, autorización, reglas de detección y endpoints críticos; agregar pruebas de carga para validar los objetivos de rendimiento.
- Validar y recalibrar los umbrales con datos representativos antes de usar el detector en un entorno real.
- Agregar trazabilidad administrativa de quién revisa o resuelve alertas si el sistema evoluciona a un entorno institucional.
- Actualizar este SRS cuando se incorporen nuevas reglas, integración real con una fuente empresarial, cambios de roles o modificaciones sustanciales en la arquitectura.

BIBLIOGRAFÍA

Pressman, R. S., & Maxim, B. R. Ingeniería del software: un enfoque práctico. McGraw-Hill.

Sommerville, I. (2016). Software Engineering. Pearson.

ISO/IEC 25010:2023. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model.

Congreso de la República del Perú. Ley N.º 29733, Ley de Protección de Datos Personales.

Ministerio de Justicia y Derechos Humanos. D.S. N.º 016-2024-JUS, Reglamento de la Ley N.º 29733.

Congreso de la República del Perú. Ley N.º 30096, Ley de Delitos Informáticos.

WEBGRAFÍA

FastAPI Documentation: <https://fastapi.tiangolo.com/>

Python 3.12 Documentation: <https://docs.python.org/3.12/>

MongoDB Documentation: <https://www.mongodb.com/docs/>

PyMongo Documentation: <https://www.mongodb.com/docs/languages/python/pymongo-driver/>

Chart.js Documentation: <https://www.chartjs.org/docs/latest/>

GitHub Documentation: <https://docs.github.com/>